Лекція №10 (2 години). Функції, замикання та контекст виконання (this).

План лекції

1. Функції як об'єкти першого класу у JavaScript.
2. Способи створення функцій: Declaration, Expression, Arrow Functions.
3. Параметри за замовчуванням та оператор Rest у функціях.
4. Лексичне оточення (Lexical Environment) та область видимості.
5. Замикання (Closures): концепція та практичне застосування.
6. Контекст виконання (this) у звичайних та стрілочних функціях.
7. Методи `call`, `apply` та `bind`.

Перелік умовних скорочень

- **JS** — JavaScript.
- **ES6** (ECMAScript 2015) — ключовий стандарт JavaScript.
- **IIFE** (Immediately Invoked Function Expression) — функція, що викликається негайно (історичний патерн).
- **Callback** — функція зворотного виклику (функція, передана як аргумент іншій функції).

Вступ

Функції є "робочими конячками" будь-якої програми. Вони дозволяють ізолювати шматок коду, дати йому ім'я та викликати його у потрібний момент скільки завгодно разів, замість того, щоб копіювати та вставляти однакові рядки. Це принцип **DRY** (Don't Repeat Yourself).

Однак у JavaScript функції — це набагато більше, ніж просто підпрограми. Вони є **об'єктами першого класу** (First-class citizens). Це означає, що функцію можна зберегти у змінну, передати іншій функції як аргумент (породивши **Callback**) або навіть повернути з іншої функції як результат (породивши **Замикання**). Ця особливість робить JS надзвичайно гнучкою мовою програмування, стираючи межі між даними та поведінкою.

У цій лекції ми розберемо еволюцію синтаксису функцій (від Function Declaration до сучасних стрілочних функцій ES6), дослідимо магію Замикань, яка часто є найскладнішою темою на співбесідах, та остаточно усунемо плутанину з ключовим словом `this`.

1. Функції як об'єкти першого класу

Якщо функція в JS є об'єктом, то ми можемо робити з нею все те саме, що і зі звичайними змінними:

- Присвоювати масивам або властивостям інших об'єктів (тоді функція називається **методом**).
- Передавати як аргументи (Callback-функції — основа роботи з подіями та асинхронністю).
- Додавати до самої функції власні властивості (наприклад, `myFunction.count = 0`).

```
// Функтив як змінна (Callback)
function greet(name) {
  console.log(`Привіт, ${name}!`);
}

function processUserInput(callback) {
  const name = prompt("Будь ласка, введіть своє ім'я.");
  callback(name); // Викликаємо функцію, яку нам передали
}

// Передаємо функцію greet як аргумент (БЕЗ ДУЖОК!)
processUserInput(greet);
```

2. Способи створення функцій

У JS існує три основних способи створення функцій, кожен з яких має свої нюанси.

2.1. Function Declaration (Оголошення функції)

Це "класичний" спосіб. Головна особливість — **Hoisting (Підняття)**. JS-рушій "знаходить" ці функції ще до початку виконання коду. Тому функцію можна викликати *до* місця, де вона написана в коді.

```
sayHi(); // Працює ідеально, хоча функція написана нижче

function sayHi() {
  console.log("Привіт!");
}
```

2.2. Function Expression (Функціональний вираз)

Функція створюється і відразу присвоюється змінній (`const` або `let`). Такі функції **не піднімаються**. Виклик такої функції до її оголошення призведе до помилки (ReferenceError). Вони часто бувають анонімними (не мають імені після ключового слова `function`).

```
// sayHello(); // ОШИБКА! Cannot access 'sayHello' before initialization

const sayHello = function () {
  console.log("Hello!");
};

sayHello(); // Працює
```

2.3. Arrow Functions (Стрілочні функції ES6)

Це найсучасніший, найкоротший і зараз найпопулярніший спосіб. Використовує "товсту стрілку" (`=>`). Стрілочні функції є анонімними (тому їх завжди присвоюють у змінну або передають як callback).

Синтаксис:

```
// Звичайна функція
const add = function (a, b) {
  return a + b;
};

// Стрілочна функція (абсолютний аналог)
const addArrow = (a, b) => {
  return a + b;
};

// Найкоротший синтаксис (Явного return немає, він відбувається АВТОМАТИЧНО)
// Працює тільки якщо тіло функції складається з ОДНОГО виразу
const addShort = (a, b) => a + b;

// Якщо параметр лише один, дужки можна опустити
const double = (x) => x * 2;
```

> ***Увага:*** *Крім короткого синтаксису, стрілочні функції мають принципову відмінність у роботі з ключовим словом* `this` *(див. розділ 6).*

3. Параметри за замовчуванням та оператор Rest

Параметри за замовчуванням (Default Parameters): Дозволяють функції мати значення "про всяк випадок", якщо при виклику цей аргумент забудуть передати (щоб він не дорівнював `undefined`).

```
function greetUser(name = "Гість", role = "Користувач") {
  console.log(`Привіт, ${name}. Твоя роль: ${role}`);
}

greetUser("Іван"); // Привіт, Іван. Твоя роль: Користувач
greetUser(); // Привіт, Гість. Твоя роль: Користувач
```

Оператор Rest (`...`): Виглядає так само, як Spread оператор (три крапки), але робить протилежне. Якщо Spread "розсипає" масив, то Rest (залишок) — "збирає" всі передані аргументи в один масив. Використовується в параметрах функції, коли ми не знаємо, скільки точно аргументів нам передадуть.

```
function sum(...numbers) {
  // Тепер numbers — це справжній масив усіх переданих чисел [1, 2, 3, 4]
  let total = 0;
  for (let num of numbers) {
    total += num;
  }
  return total;
}

console.log(sum(1, 2)); // 3
console.log(sum(1, 2, 3, 4)); // 10
```

4. Область видимості (Scope) та Лексичне оточення

Будь-яка змінна у JS існує в певному середовищі — **області видимості**.

1. **Глобальна область видимості:** Змінні, оголошені поза будь-якою функцією чи блоком. Доступні з будь-якої точки коду.
2. **Блочна область видимості (`let`, `const`):** Змінна заперта всередині фігурних дужок `{ ... }` (в циклі `for`, в `if`, всередині функції). Зовні її не видно.
3. **Функціональна область видимості (`var`):** Заперта лише всередині функції, ігнорує звичайні блоки `if` або `for`. Це причина не використовувати `var`.

Лексичне оточення (Lexical Environment): Коли створюється функція, вона запам'ятовує, де (в якому місці коду) вона народилася. Лексичне оточення — це прихований об'єкт, який створюється при виклику функції. Він містить:

- Всі локальні змінні цієї функції.
- Посилання на зовнішнє лексичне оточення (тобто змінні "батьківського" блоку, де функція була створена).

Правило пошуку: Якщо скрипт не знаходить змінну всередині функції, він "йде наверх" — шукає її в зовнішньому оточенні, і так аж до самого глобального об'єкта `window`. Це називається "Ланцюгом областей видимості" (Scope Chain).

5. Замикання (Closures)

Замикання — це здатність функції запам'ятовувати своє зовнішнє лексичне оточення і мати доступ до нього, навіть коли функція викликається ПОЗА цим оточенням.

Замикання виникає щоразу, коли функція створюється всередині іншої функції і повертається з неї.

```
function createCounter() {
  let count = 0; // Локальна змінна батьківської функції

  // Повертаємо внутрішню функцію як результат
  return function increment() {
    count++; // Внутрішня функція використовує count із зовнішньої функції
    console.log(count);
  };
}

const myCounter = createCounter(); // createCounter відпрацювала і завершилась
const counter2 = createCounter(); // Створили зовсім незалежний лічильник

// Але внутрішня функція (в myCounter) ЗАПАМ'ЯТАЛА змінну count!
myCounter(); // 1
myCounter(); // 2
myCounter(); // 3

counter2(); // 1 (він має своє особисте замикання)
```

Навіщо потрібні замикання?

1. **Інкапсуляція:** Ми надійно сховали змінну `count` у фрагменті коду вище. Жодним способом ззовні програми ми не можемо напряму змінити `count` (наприклад `count = 100`).

Єдиний спосіб впливати на неї — викликати нашу функцію `myCounter()`. Це основа створення "приватних" змінних у JS.

2. Програмування подій, функцій зворотного виклику (Callbacks) та часткове застосування функцій (Каррування).

6. Контекст виконання (this)

Ключове слово `this` (Цей) — найзаплутаніша концепція в JS для початківців. Загальне правило: `this` посилається на об'єкт, **який зараз викликає функцію**. Значення `this` обчислюється не там, де функція написана, а В МОМЕНТ її виклику.

Ситуація 1: Метод об'єкта Якщо функція викликається як метод (через крапку: `object.method()`), то `this` — це об'єкт перед крапкою.

```
const user = {
  name: "Петро",
  sayHi() {
    console.log(`Привіт, я ${this.name}`); // this вказує на об'єкт user
  },
};
user.sayHi(); // Викличе "Привіт, я Петро"
```

Ситуація 2: Простий виклик функції Якщо ви просто викликаєте звичайну функцію (без крапки), `this` вказує на глобальний об'єкт (в браузері це `window`). Якщо увімкнено суворий режим (`"use strict";`), то `this` буде дорівнювати `undefined`.

Ситуація 3: Стрілочні функції (Arrow Functions) Стрілочні функції **не мають власного** `this`. Вони беруть (запозичують) `this` з навколишнього середовища (з того контексту, де вони були НАПИСАНІ). Це їхня головна відмінність.

Коли це корисно? Коли ми створюємо callback-функції всередині методів об'єкта. Звичайна функція "загубила" б контекст, а стрілочна його утримає.

```
const group = {
  title: "Група 101",
  students: ["Іван", "Марія"],

  showList() {
    // Використовуємо стрілочну функцію у forEach
    // Вона не має свого this, тому бере this батька (методу showList), тобто
```

```
об'єкт group.
    this.students.forEach((student) => {
      console.log(`${this.title}: ${student}`);
    });
  },
};
group.showList();
// Група 101: Іван
// Група 101: Марія
// Якби у forEach була звичайна function(student){...} - ми б отримали undefined:
Іван
```

> ***Хінт:*** *Ніколи не використовуйте стрілочні функції як методи об'єкта!*

7. Примусове прив'язування: call, apply, bind

Іноді нам потрібно взяти метод з одного об'єкта і застосувати до іншого, примусово вказавши, чим має бути `this`. Для цього у кожної звичайної функції є вбудовані методи.

1. **`call(context, arg1, arg2...)`**: Викликає функцію *негайно*, підміняючи `this` на `context`, а аргументи передаються через кому.
2. **`apply(context, [array_of_args])`**: Робить те саме, що й `call`, але приймає аргументи як один єдиний масив.
3. **`bind(context, arg1...)`**: Вона *не викликає* функцію негайно. Вона повертає **нову** функцію, яка назавжди "прив'язана" до вказаного `this`. Стандартно використовується в React для методів класів.

```
const animal = { sound: "Роар!" };

function speak(msg) {
  console.log(`${msg} Каже: ${this.sound}`);
}

// Примусово вказуємо, що this має дорівнювати об'єкту animal
speak.call(animal, "Увага!"); // Увага! Каже: Роар!

// Створюємо нову функцію, намертво прив'язану до animal
const boundSpeak = speak.bind(animal);
boundSpeak("Привіт!"); // Привіт! Каже: Роар!
```

Висновки

1. У JS функції — це об'єкти вищого порядку (First-class citizens). Їх можна передавати туди-сюди як змінні.
2. Стрілочні функції (`() => {}`) надають короткий синтаксис та не створюють власний контекст `this` , що рятує при використанні Callbacks.
3. Оператор Rest (`...`) дозволяє елегантно збирати змінну кількість аргументів функції в єдиний масив.
4. Замикання (Closures) — одна з найважливіших концепцій. Функція назавжди запам'ятовує змінні того місця (лексичного середовища), де вона була написана. Це використовується для приховування даних.
5. Правило `this` : у звичайній функції `this` визначається тим, **ЯК** функція була викликана (об'єкт перед крапкою). У стрілочній функції `this` береться з того місця, **ДЕ** функція була написана (Лексичний `this`).
6. Методи `call` , `apply` та `bind` дозволяють програмісту стати диктатором і явно вказати функції, що саме має бути її контекстом `this` .

Джерела

1. MDN Web Docs: Functions
2. MDN Web Docs: Closures
3. Javascript.info: Замикання та лексичне оточення
4. Javascript.info: Arrow functions revisited

Запитання для самоперевірки

1. Чим відрізняється Function Declaration від Function Expression?
2. В чому специфіка стрілочних функцій (Arrow Functions) окрім їхнього короткого синтаксису? Чого вони "НЕ мають"?
3. В чому різниця між Rest оператором та Spread оператором, коли обидва виглядають як `...` ?
4. Опишіть своїми словами, що таке Замикання (Closure). Для якої найпопулярнішої проблеми його використовують?
5. Що виведе `console.log(this)` в глобальній області видимості в браузері? А всередині методу об'єкта?
6. Якщо у вас є метод об'єкта `user.showName()` , і ви передасте його як callback (наприклад у таймер: `setTimeout(user.showName, 1000)`), швидше за все контекст загубиться. Як вижити в цій ситуації за допомогою методів `bind` або стрілочної функції?
7. У чому принципова різниця між методами функції `call()` та `bind()` ?